

CS 636 Semester 2024–2025-II: Assignment 2

23rd March 2025

Due Your assignment is due by Apr 12, 2025, 11:59 PM IST.

General Policies

- You should do this assignment ALONE.
- Do not copy or turn in solutions from other sources. You will be PENALIZED if caught.

Submission

- Submission will be through Canvas.
- Submit a compressed file called “`<roll>-assign1.tar.gz`”. The compressed file should have the following structure.

```
-- roll-assign2
-- -- roll-assign2.pdf
-- -- Makefile
-- -- <problem1-dir>
-- -- -- <roll.cpp>
-- -- <problem2-dir>
-- -- -- <roll.cpp>
-- -- <problem3-dir>
-- -- -- <roll.cpp>
```

Name the `.cpp` and `.h` source files using your roll number.

Use the `Makefile` for compilation. An example compilation command can be `make problem1`.

Use the PDF file to include details (e.g., compilation command and flags, optimizations, challenges, and results) about your solution.

- We encourage you to use the L^AT_EX typesetting system for generating the PDF file. You can use tools like Tikz, Inkscape, or Draw.io for drawing figures if required. You can alternatively upload a scanned copy of a handwritten solution, but MAKE SURE the submission is legible.
- You can optionally use the [Google Test framework](#) for testing your data structure implementations. You can add support in the `Makefile` with `make test`.
- You will get up to TWO LATE days to submit your assignment, with a 25% penalty for each day.

Evaluation

- Write your programs such that the EXACT output format (if any) is respected.
- We will evaluate your implementations on a Unix-like system, for example, recent Debian-based distributions installed on KD first floor labs.
- We will evaluate the implementations with our inputs and test cases, so remember to test thoroughly.

Problem 1

[70 marks]

Implement a concurrent closed-chaining-based hash table using Pthreads. Duplicate keys are not allowed in the hash table. Implement support for the following operations.

Listing 1: Operations supported by the bounded partial queue data structure

```
1 void batch_insert(HashTable *ht, KeyValuePairs *kv_pairs, bool* result);
2 void batch_delete(HashTable *ht, KeyList *key_list, bool* result);
3 void batch_lookup(HashTable *ht, KeyList *key_list, uint32_t* result);
```

The second parameter in the above prototypes should be self-explanatory based on the type name. Key and value are four bytes (unsigned int) each. The third parameter **result** is a boolean array that stores the result of each insert or delete query. For **batch_lookup()**, **result** will contain the values if present. Use zero to indicate a failed search. Adapt the driver we have shared to the exact prototype that you design.

Description

- The APIs work on a batch of items (size of the batch can be one). Each batch contains of operations of the same type. For example, there can be concurrent insertions of 10000 key and value pairs in a batch, followed by a batch that tries to delete 300 keys.
- Assume that the concurrency is limited to operations in a batch. Use threading to ISSUE concurrent calls to the hash table data structure (e.g., OpenMP). You can spawn n threads and distribute the work among the n threads, and then the threads run concurrently issuing calls to the concurrent data structure.
- Use the following traces as the source for random sequences of keys and values.
 - [random_keys_insert.bin](#)
 - [random_values_insert.bin](#)
 - [random_keys_delete.bin](#)
 - [random_keys_search.bin](#)
- Ensure the empirical correctness of your code with unit tests that contain a fixed sequence of operations. Include the test cases as separate function calls (e.g., **test1** and **test2**). You can compare the results of your **batch_insert()** kernel by iteratively checking that all inserted keys and values are indeed present.
- The [TBB library](#) from Intel implements a closed-chaining-based hash table. TBB's implementation is open source and should be installed under `/usr/include/tbb` on the KD workstations. Use any version that is installed and works for you (either legacy TBB or oneTBB).
- Use compile time flag **USE_TBB** to switch between the two versions. Add support via the **Makefile**.
- Invoking **malloc** frequently is poor for performance. You can explore optimizations to avoid invoking **malloc** frequently.

Submission

- (i) A header file that implements the hash table,
- (ii) Extend the driver code to make concurrent calls to the hash table,
- (iii) Implement a `print()` method which can be invoked after every operation (e.g., `batch_insert()`) to check the content of the hash table,
- (iv) Report the time taken and the throughput (e.g., 1000 inserts per second) to complete n concurrent operations, where $n \in \{1e5, 1e6, 1e7\}$,
- (v) Extend the driver to use the concurrent hash table provided by Intel TBB,
- (vi) Compare the throughput of the TBB and your Pthread implementations.

Problem 2

[50 marks]

Implement an “unbounded, total, lock-free” concurrent queue using linked lists. The queue will contain only positive 4-byte integer values.

Listing 2: Operations supported by the unbounded total lock-free queue data structure

```
1 bool enq(int v);  
2 int deq();
```

Since the queue is unbounded and total, we do not need APIs like `isEmpty()` and `isFull()`.

Description

- Your implementation should be non-blocking, i.e., you should NOT use locks.
- You can maintain head and tail pointers using `std::atomic<T>`.
- Ignore OOM issues while implementing the `enq()` operation.
- A `deq()` operation on an empty queue should return a negative integer, i.e., it will not block.
- Use `random_values_insert.bin` from Problem 1 to generate a sequence of random values to be pushed.
- Whether a thread issues a `enq()` or a `deq()` can be decided based on probability. We are not using batched kernel operations for this problem.
- Lock-free implementations are vulnerable to the ABA problem. Avoid ABA in your solution using CAS operations on pointers encoded with a counter. Using separate fields for the pointer and the counter does NOT prevent ABA unless they are adjacent in memory and your architecture provides a 128-bit CAS.

Submission

- (i) A header file that implements the concurrent queue,
- (ii) Extend the driver code from Problem 1 to issue concurrent calls to the queue,
- (iii) Implement a `print()` method which can be invoked from the driver to check the content of the queue,
- (iv) Report the time taken to complete n concurrent operations (i.e., a mix of `enq()` and `deq()`), where $n \in \{1e5, 1e6, 1e7\}$,
- (v) There are different implementations of concurrent queues. You can check Intel TBB, [moody-camel::ConcurrentQueue](#), and Boost. Extend the driver to use and compare with the [lock-free queue class from Boost library](#),
- (vi) Evaluate strong scalability (i.e., workload remains the same) of your implementation with 1,2,4,8, and 16 threads.

Problem 3

[50 marks]

Implement a concurrent [Bloom filter](#) for tracking 4-byte unsigned integers. Your filter should be of size 2^{24} and use three hash functions of your choice.

Listing 3: Operations supported by the Bloom filter data structure

```
1 void add(int v);  
2 bool contains(int v);
```

- You are free to use sophisticated hash functions of your choice to reduce the chances of false positives. However, the hash computation will impact your implementations performance.
- Use threading to issue concurrent calls to the Bloom filter data structure.
- Use [random_keys_insert.bin](#) from Problem 1 to generate a sequence of random values to be added.
- Whether a thread issues a `add()` or a `contains()` can be decided based on probability.
- Ensure empirical correctness of your code with unit tests that contain a fixed sequence of operations. Include the test cases as separate function calls (e.g., `test1` and `test2`).

Submission

- A header file that implements the Bloom filter,
- Extend the driver code to make concurrent calls to the Bloom filter,
- Implement a `print()` method which can be invoked after every operation (e.g., `add()`) to check the content of the filter,
- Report the time taken to complete n concurrent operations, where $n \in \{1e5, 1e6, 1e7\}$,

- Report the false positive (i.e., the number has not been added but `contains()` returns true) rate for each input size,
- Evaluate strong scalability (i.e., workload remains the same) of your implementation with 1,2,4,8, and 16 threads.